

Learning Journey Assistant Sprint Plan — Sprints A, B and C

CSE5IDP Industry Development Project · Semester 2, 2026 · Group 3 · Jira project **IOLG**

Project owner: Dr Scott Mann · Repository: github.com/yetanotherpassword/LearningJourneyAssistant

Revision 3, 24 August 2026 — supersedes [docs/sprint-plan.md §4–§7](#)

This document contains one **binding** sprint and two **indicative** ones. Sprint A is fully specified and is what gets entered into Jira now. Sprints B and C are recorded so the team can see the shape of the remaining work and so dependencies are visible early — they are *not* entered into Jira yet, because committing to detail eight weeks out is guesswork dressed up as planning. They get refined at the end of the sprint before them.

1. Agile and Jira terminology, as this project uses it

Jira gives you several issue types and it is not obvious which to use for what. This is the mapping this project uses. Use it consistently — an inconsistent board produces a meaningless burndown, and the burndown is a graded artefact.

Term	What it means	In this project
Product backlog	The ordered list of everything the product might eventually need. Living, never finished.	Everything in this document plus the nice-to-haves from the proposal. Lives in Jira, ordered by priority.
Sprint backlog	The subset the team commits to for the current sprint only. Stable once the sprint starts.	The eleven Sprint A items in §5. New requests during the sprint go to the product backlog, not into Sprint A.
Epic	A large body of work spanning multiple sprints. A container, never worked on directly.	Seven epics, §4. Each maps to a proposal must-have. You create these once and then attach everything else to them.
Story	One user-facing slice of value. Fits inside one sprint. Written as "as a <role> I want <action> so that <value>".	Anything a student, staff reviewer or the project owner would notice — a dashboard view, the confirmation gate, a generated learning plan.
Task	Engineering work that is necessary but not user-facing. Same size rules as a story.	CI setup, landing code in the repo, an architecture decision record. Nobody "uses" CI, but it must be done.
Sub-task	A slice of a story or task, small enough to assign to one person and finish in a day or two.	The five individual pull requests inside A1. This is the level where individual names attach.
Bug	A defect found in work already called done.	Created as found, attached to the epic that owns the broken component.
Story points	A relative measure of effort and complexity, not hours. Fibonacci (1, 2, 3, 5, 8, 13) because large estimates are inherently less precise.	Every story and task carries points. Sub-tasks do not — their parent already carries the estimate, and pointing both double-counts the burndown.
Definition of Ready	The bar an item must clear before it can be pulled into a sprint.	§6. Acceptance criteria written, points estimated, owner identified, dependencies known.
Definition of Done	The bar an item must clear before it counts as complete.	§6. Merged via reviewed PR, tests pass, README current, demoable.

The distinction that matters most

An epic is *not* a big task. It is a theme that stories hang off. "Mastery Dashboard" is an epic; "Add strengths view" is a story inside it. If you find yourself assigning an epic to a person and estimating it, you have built a task and labelled it wrongly.

2. Team and ownership — fill this in before entering anything

The plan uses owner slots M1–M5. These have been unassigned since Sprint 1 and every table in this document is unusable until real names are attached. Do this at Sprint A planning, in the session, out loud.

Slot	Area of ownership	Primary paths	Name
M1	Moodle integration & extraction	lja/extraction/ , sql/ , devenv liaison	_____
M2	Data model & gap engine	lja/model/ , lja/data/	_____
M3	AI / LLM layer	lja/llm/ , lja/generate/	_____
M4	Dashboard	lja/dashboard/ , the four mastery views, UX	_____
M5	DevOps, QA, security, docs	devenv/ , CI, testing, compliance, docs/	_____

Two constraints on how you fill it in. First, whoever currently holds the uncommitted lja/ codebase must take M2, because A1 (landing that code) can only be done by the person who has it. Second, ownership means "the person who says yes or no on pull requests in this area and carries the pager for its bugs" — not "the only person allowed to edit it".

Unresolved: how many people are on this team?

docs/sprint-plan.md §1 says five and rosters five scrum masters. devenv/env.sh says "all six of us". These cannot both be right, and the answer changes how many scrum-master presentations are required. Resolve it with Ali before Sprint A planning and fix whichever file is wrong in the same commit.

3. Calendar

Sprint	Dates	Capacity	Subject context
A	Mon 24 Aug – Sun 6 Sep	≈70%	Mid-semester break falls in week 2, from 31 Aug. Scope is deliberately light. Take the days off — that is the lecturer's own instruction.
B	Mon 7 Sep – Sun 20 Sep	100%	Sessions 8–9. Effective reporting, product handover.
C	Mon 21 Sep – Sun 4 Oct	100%	Sessions 10–11. Feature freeze declared at the Sprint C review.
—	Mon 5 Oct – Sun 18 Oct	—	Not a delivery sprint. Session 12 (implementation report, sprint compendium), session 13 (final presentation, 12 Oct), final submission Sunday 18 October.

Sprint A carries 38 points against Sprint B and C's 50. That gap is the mid-semester break, at roughly 74% of full capacity. Planning equal points across all three would mean pretending the break does not exist, which produces exactly the flat-then-cliff burndown shape the lecturer has said he looks for and asks about.

Raise with Ali this week: the scrum-master count

Three delivery sprints plus Sprint 1 gives **four** scrum-master presentations. The subject requires six, one per member. Preferred fix: run 5–18 October as a ceremonial Sprint D whose backlog is the implementation report, the compendium, the handover and the final presentation. It costs no engineering time because that work exists anyway, but it still carries a scrum master, ceremonies, a burndown and a graded presentation. Do not assume four is acceptable — get it confirmed in writing in the Teams channel.

4. Epic structure

Create these seven epics first, before any story. Everything else in this document attaches to one of them. Epics are not assigned to individuals and carry no story points.

Ref	Epic name	Covers
E1	Engineering Foundations	Version control, CI, branch protection, review process. Not a proposal must-have, but everything else is unreviewable without it.
E2	Moodle Integration & Extraction	Proposal must-have "secure Moodle integration". The devenv, the Web Services probe, the SQL extraction path.
E3	Competency Model & Gap Engine	Proposal must-haves "competency model aligned with learning outcomes" and "parsing engine for assessment rubrics and feedback". SILO clustering, gap detection, the staff confirmation gate, feedback analysis.
E4	Mastery Dashboard	Proposal must-have "visual mastery dashboard" and its four views: current understanding, strengths, skill gaps, progress trends.
E5	Grounded Generation	Proposal must-haves "personalised learning plan generation" and "evidence-based study strategy recommendations", plus the grounding infrastructure that keeps them honest.
E6	Research Export	Export of attainment, gap and plan data in a stable versioned format for the department's longitudinal comparison study.
E7	Compliance, Security & Handover	Compliance checklist, security passes, measured non-functional requirements, handover document, runbook, demo rehearsal.

5. Sprint A — binding · Mon 24 Aug - Sun 6 Sep · 38 points

Sprint goal

Any team member can clone the repository and run the whole pipeline on their own machine, and no gap report can be produced from unreviewed LLM clustering without saying so.

Two things sit behind that goal. The working code — the LLM layer, the Excel loader, the SILO clustering, gap detection, the CLI, the dashboard and 54 tests — is *not in the repository*. It exists on one machine while every README describes it as built. That is a bus factor of one, it makes the team's own Definition of Done impossible to meet, and it means a clone by the project owner produces documentation for software that is not there. And the clustering that drives every gap finding is unreviewed LLM output with nothing marking it as such.

Sprint A backlog

Ref	Summary	Type	Epic	Owner	Pts	Acceptance criteria
A1	Land the LJA Python package and test suite in <code>main</code>	Task	E1	M2	5	Five separate PRs, one per sub-task. Full-history secret scan clean before the first push. <code>output/</code> and <code>.env</code> confirmed ignored. Accepted only when a different team member clones fresh, creates the conda environment, runs <code>python -m lja.cli</code> against the dataset and <code>pytest</code>, and both pass — on their own machine, unassisted.
A2	Peer review the five code-landing pull requests	Task	E1	team	5	Five sub-tasks, claimed at planning. Every member except the author reviews at least one. Review means checking out the branch and running it, not approving on sight. Reviewer comments recorded in the PR, not in Teams.
A3	Set up CI and branch protection on <code>main</code>	Task	E1	M5	3	GitHub Actions running <code>ruff</code> and <code>pytest</code> on every PR to <code>main</code> . Branch protection: no direct pushes, one approving review required, CI must pass before merge. Status badge in the root README. Verified by a deliberately failing test being blocked.
A4	Merge or close the two pull requests open since 16 August	Task	E1	M1	2	<code>docs/macos-setup-readme</code> and <code>Istiaque/Tender-Document</code> each either merged into <code>main</code> or closed with a written reason. No branch left stranded — a tender document living only on a personal branch is not part of the handover.

Ref	Summary	Type	Epic	Owner	Pts	Acceptance criteria
A5	Add a staff confirmation gate for LLM SILO clustering	Story	E3	M1 (paired with M2)	5	Three states per competency cluster: <code>pending</code> , <code>confirmed</code> , <code>rejected</code> . Persisted <i>alongside</i> <code>output/silo_clustering.json</code> , not inside it — the cache is regenerable, the review decisions are not. <code>python -m lja.review</code> lists clusters with their SILO members, sets state, attaches a note. <code>lja.cli</code> warns when a gap report is driven by <code>pending</code> clustering; <code>--allow-unconfirmed</code> required in scripts. A <code>rejected</code> cluster routes to a documented rework path, never a dead end. Unit test per state transition.
A6	Add the "current understanding level" dashboard view	Story	E4	M4	3	Per-student mastery across all competencies, not only the gaps. Rendered from existing <code>compute_gaps()</code> output — no new pipeline stage. Reuses the semantic palette in <code>dashboard/static/style.css</code> so chart and badge colours cannot disagree.
A7	Add the "strengths" dashboard view	Story	E4	M4	3	High-attainment competencies as a first-class view, threshold-driven from A9's configuration rather than a second hardcoded number. Test coverage via the existing in-memory dashboard fixtures.
A8	Extract a reusable grounding validator	Task	E5	M3	5	Generalise <code>_validate_coverage()</code> into <code>lja/llm/grounding.py</code> — a checker any generated artefact can be validated against, not just clustering. Unblocks every generation feature in Sprints B and C. Existing clustering tests still pass against the extracted version.
A9	Make gap thresholds configurable from a single source	Task	E3	M2	2	The 50 and 65 values currently hardcoded in both <code>gap_detection.py</code> and SQL query 6 come from one configuration source. No literal threshold left in either file. Existing tests parameterised rather than pinned to the old constants.
A10	Close the competency-mechanism architecture decision record	Task	E2	M5	3	Time-boxed to one day, hard stop. Load <code>competency_framework_cse5idp.csv</code> into the devenv Moodle, run SQL queries 3 and 4, record which returns rows in <code>sql/README.md</code> with the output that justifies it. If neither returns rows, record that — it makes the bridging table the product rather than a workaround, which is a defensible architectural claim for the design report.

Ref	Summary	Type	Epic	Owner	Pts	Acceptance criteria
A11	Verify the Anthropic client against a live API key	Task	E5	M3	2	The <code>AnthropicClient</code> path has never been run — only the local Ollama path has. One live structured-output call succeeds end to end. Result and any surprises recorded in <code>python/README.md</code> . No key committed.

Sub-tasks for A1 and A2

These are the only items in Sprint A that decompose. Sub-tasks carry no story points — the parent already does — but they do carry individual names, which is where evidence of individual contribution comes from.

Parent	Sub-task	Assignee
A1	PR (a) — <code>lja/config.py</code> and <code>lja/llm/</code>	M2
A1	PR (b) — <code>lja/data/</code>	M2
A1	PR (c) — <code>lja/model/</code>	M2
A1	PR (d) — <code>lja/dashboard/</code>	M2
A1	PR (e) — <code>lja/cli.py</code> and <code>tests/</code>	M2
A2	Review PR (a)	claim at planning
A2	Review PR (b)	claim at planning
A2	Review PR (c)	claim at planning
A2	Review PR (d)	claim at planning
A2	Review PR (e)	claim at planning

Workload distribution

Owner	Items	Points
M1	A5 (5) · A4 (2) · one A2 review (1)	8
M2	A1 (5) · A9 (2)	7
M3	A8 (5) · A11 (2) · one A2 review (1)	8
M4	A6 (3) · A7 (3) · one A2 review (1)	7
M5	A3 (3) · A10 (3) · two A2 reviews (2)	8
Sprint A total		38

Two reallocations make that even, and both are deliberate. **M1 pairs on the confirmation gate** even though it is model-layer work outside their area — the team's bus factor is one, and pairing the least-loaded member with the sole code author on a real feature is the cheapest knowledge transfer available. **M5 takes the competency ADR** rather than M1, because it runs against the devenv Moodle that M5 owns, and because M1 is already carrying the gate.

If the break costs more time than planned

Drop A7 first, then A10. **A1 through A5 are non-negotiable** — without them the sprint goal is not met regardless of what else ships, and the two after them are what make the goal demonstrable.

Two decisions to put to Scott inside Sprint A

Both block Sprint B stories. Raise them in the first sprint review at the latest, and preferably in the Teams channel this week, because the project owner has a day job and may take a week to answer.

1. **Trajectory semantics.** The supplied workbook has no dates and no attempt timestamps anywhere — 1650 rows, 11 assessments, no time dimension at all. "Progress trends", the fourth dashboard view in the proposal, can therefore only mean progression across year levels (CSE10OF → CSE2ALG → CSE3CAP), not a time series. Confirm that is the intended reading before B3 is built on it, and make sure the final report says which it is.
2. **Export field set.** Confirm what the department's longitudinal comparison actually needs per record before B10 defines the schema. Changing an export schema after data has been generated against it is expensive, and the whole point of the export is that runs from different points in time are comparable.

Sprint A review demo

A team member who did *not* write the code clones the repository on their own machine and drives everything: the CLI run, three dashboard views, a cluster moved from `pending` to `confirmed` and the badge changing to match. **A demo on the author's laptop does not meet the sprint goal**, however well it runs.

6. Definition of Ready and Definition of Done

Definition of Ready — before an item enters a sprint

- Acceptance criteria written, in testable terms, not aspirations.
- Story points estimated by the team, not by the author alone.
- An owner identified, and the owner agrees.
- Dependencies on other items or on a project-owner decision are known and recorded.
- Attached to an epic.

Definition of Done — before an item is closed

- Merged into `main` via a pull request, reviewed by a **different** owner than the author.
- `pytest` passes; new logic carries unit test coverage.
- The relevant bundle README updated if setup or behaviour changed — not deferred to "docs later".
- No secrets committed; `.env.example` updated if new configuration was added.
- **Demoable** — reachable from the dashboard or a documented CLI entry point. Code that exists only as an unused function is not done.
- The Jira issue moved to Done *as the work completes*, not the night before the presentation.

7. Sprint B — indicative · Mon 7 Sep - Sun 20 Sep · 50 points

Not entered into Jira yet. Refine and commit at the Sprint A retrospective.

Sprint goal

All four proposal dashboard views are live, gap findings cite what the marker actually wrote, the first student-facing generated artefact exists and is provably grounded, and data can leave the system in a stable format.

Ref	Summary	Epic	Owner	Pts	Note
B1	Moodle vertical slice, time-boxed to this sprint	E2	M1	8	One subject, seeded rubric fillings, SQL query 2 through a Python reader into <code>lja_criterion_score</code> , consumed by the same <code>compute_gaps()</code> . If the Moodle path needs its own gap engine, the abstraction is wrong and that is worth finding out now. Hard stop at the sprint boundary — no carry into C.
B2	Table prefix as configuration, not a constant	E2	M1	2	Our own two environments already disagree — <code>mdl_</code> versus <code>devenv's m_</code> . A hardcoded prefix is a known defect waiting to happen.
B3	Trajectory model	E3	M2	5	Attainment per competency ordered along a <i>declared</i> subject-sequence configuration, not parsed out of the course code. The no-timestamps constraint stated in the module docstring and surfaced on the page.
B4	Feedback attribution layer	E3	M2	5	Every finding points at a source row and a character span; a paraphrase is never presented as fact. Degrades cleanly when the comment is one of the 45 templated strings, which most are. Closes the proposal objective the pipeline currently skips entirely.
B5	Learning-plan generation with grounding tests	E5	M3	8	Grounding tests are part of the story, not a follow-up. Via A8, every SILO, subject, assessment and competency name in the output must appear in the input, or the build fails.
B6	Structured-extraction schema and prompt for B4	E3	M3	2	Reuses A8's validator.
B7	Progress-trends dashboard view	E4	M4	5	Completes all four proposal views.
B8	Feedback evidence panel under each gap	E4	M4	3	Renders B4's attributed feedback with its source visible.
B9	Dev-mode student selector	E4	M4	2	Full authentication stays out of scope and is recorded as a known limitation.
B10	Export layer	E6	M5	8	Per-student and cohort-level attainment, gap classification, trajectory and plan, as CSV and JSON. Versioned schema, because the longitudinal comparison depends on exports from different runs being diffable. -- anonymise using salted stable pseudonyms.

Ref	Summary	Epic	Owner	Pts	Note
B11	Security pass 1	E7	M5	2	Confirm <code>lja_reader</code> is genuinely read-only, token capabilities match documentation, history clean of secrets.
Sprint B total — 10 points per person				50	

Review demo: pick a student the system flags with a persistent gap. Show the four views, the marker comments backing the finding, the generated learning plan, and the exported CSV row — live, end to end.

8. Sprint C — indicative · Mon 21 Sep – Sun 4 Oct · 50 points

Not entered into Jira yet. Refine and commit at the Sprint B retrospective.

Sprint goal

One more feature, then stop. Everything after that is evidence, hardening and handover, so the 5–18 October fortnight is free for the implementation report and the final presentation.

Ref	Summary	Epic	Owner	Pts	Note
C1	Regression pass across extraction and both data paths	E2	M1	3	
C2	Handover document	E7	M1	5	Architecture, the competency-mechanism decision and rationale, the rubric-fills-via-database caveat stated plainly — it is not API-mediated and the handover has to say so — known limitations, explicit "not built" list.
C3	Re-seed and redeploy runbook	E7	M1	2	Verified by following it on a clean machine.
C4	Threshold sensitivity chart and finalised ADR	E3	M2	5	Student counts per band across a threshold sweep. Scott has said there is no institutional "at risk" number to match, so this is the team's decision to make and defend. Quantified-NFR evidence for the report.
C5	Gap engine regression and bug bash	E3	M2	3	
C6	README truth pass — model and data bundles	E7	M2	2	Forward-looking language rewritten as what is actually true.
C7	Study-strategy recommendations	E5	M3	8	Tied to gap <i>classification</i> , so an isolated gap and a persistent cross-subject gap get structurally different advice. Every recommendation cites the competency and its evidence. Last feature built.
C8	Extend the grounding suite to cover C7	E5	M3	2	
C9	Surface plans and strategies in the dashboard	E4	M4	5	
C10	UX and consistency pass	E4	M4	3	Polish only. No redesigns this late.
C11	Dashboard README truth pass	E7	M4	2	

Ref	Summary	Epic	Owner	Pts	Note
C12	Complete docs/compliance-checklist.md	E7	M5	5	From the proposal's admin checklist — legal, security, financial, technical, HR, communication, QA, documentation. Every item closed or explicitly deferred with a written reason. Currently unwritten.
C13	Measured non-functional requirement table	E7	M5	3	Pipeline wall-clock at 150 and 300 students, LLM call count and token cost per run, dashboard render time. Numbers with method .
C14	Demo rehearsal from a clean clone	E7	M5	2	Against the running system, not slides.
Sprint C total — 10 points per person				50	

9. What was cut to fit three sprints

Compressing four sprints into three does not make the team faster. These are cut now, deliberately and on the record, rather than discovered as failures at the final review. The lecturer's stated position is that an honestly-analysed failure scores well and a fabricated success does not.

Cut	Rationale	How it is reported
Adaptive quiz generation	Pre-agreed descope priority 1 in the original plan — highest complexity-to-value ratio of the four AI features, and the one most likely to look plausible while quietly not being grounded. Cutting it in advance is worth more than cutting it under pressure in week 11.	Named in the implementation report as a planned descope, with the reasoning. Not silently omitted.
Generalised Moodle production loader	Reduced to one time-boxed vertical slice (B1). The generalised loader across all subjects is not built.	Documented design plus spike evidence, and the rubric-fills-via-database caveat stated plainly in the handover.
Confirmation UI on the dashboard	CLI review only (<code>python -m lja.review</code>). The gate itself is not cut — only its web interface.	Known limitation.
Authentication	Dev-mode student selector only, as already scoped in the base plan.	Known limitation.

Not cut under any schedule pressure: the extraction → clustering → gap-detection spine, the four dashboard views, the confirmation gate, export, and learning plans. Those are the proposal's must-haves and the department's research dependency. A working dashboard with honest gap detection is a defensible MVP on its own even if every generative feature above it is cut.

10. Process items — currently the weakest part of the project

- **Commit spread.** Three of five members have committed; one holds 15 of 19 commits. Individual contribution is assessed and `git log` is the artefact. A1's five-PR split and A2's review allocation exist partly to create that evidence. Every member should own merged work in every sprint from here.
- **One source of truth for the backlog.** The original plan specified GitHub Issues plus a Project board; neither exists. Jira (IOLG) is the graded surface, so Jira wins. Reference Jira issue keys in commit messages (`Refs IOLG-nn`) so the burndown, the ticket and the code change form one chain rather than three separate claims.

- **Burndown honesty.** Move tickets as work completes. Sprint A's ticket movement is the first burndown anyone outside the team will see. The two shapes that give the game away — the cliff and the flat line — are exactly what gets asked about, and an honestly explained bad chart costs nothing.
- **Sprint review with the project owner at every boundary,** demoed from the running system rather than slides.